

# Lecture 13: Hierarchical RL. Part II: Modern days

From explicit hierarchy to learned abstractions and pretrained planners

---

**Petr Kuderov**

Researcher at MIPT & AIRI

April 29, 2026

# Part I Was the Classical View

In the previous part we treated HRL as an explicit control architecture:

Classical HRL:

- high-level controller
- low-level controller
- options / skills / goals
- SMDP-style temporal abstraction

Main promise:

- sparse rewards handling
- long horizons
- reusable behaviors
- better exploration

This view is still useful, but it is mostly a **pre-2020 view** of the field.

Examples from Part I: Options Framework, Option-Critic, Feudal RL, FuN, HIRO, MLSH.

# What Changed?

HRL stopped being a clean, self-contained subfield.

Its core ideas diffused into:

- unsupervised skill discovery
- goal-conditioned RL
- representation learning
- world models
- offline RL
- sequence modeling
- robotics systems
- LLM / VLM agents

The label “HRL” became less dominant, but temporal abstraction became more central.

# The Central Question

Classical HRL mostly asked:

How can we learn and use high-level actions?

Modern HRL asks a broader question:

How can an agent discover useful structure across time?

This structure may appear as:

- explicit options
- latent skills
- goals in representation space
- trajectory chunks
- language plans
- tool-use programs

# Two Hard Problems Behind HRL

End-to-end HRL has to solve at least two hard problems.

## 1. Abstraction representation

What should the high-level policy see?

$$s \rightarrow z = \varphi(s)$$

The wrong abstraction makes good subgoals invisible.

## 2. Temporal chunking

When should one subgoal end and the next begin?

$$\tau = (s_t, \dots, s_{t+k})$$

The wrong chunking makes hierarchy meaningless.

HRL is not only about training two policies. It is about learning the right units of cognition and behavior.

# Why This Is Hard in RL

RL usually learns from narrow interaction data.

What the agent gets:

- one environment or benchmark suite
- limited task distribution
- sparse scalar rewards
- expensive trial-and-error

What hierarchy requires:

- reusable abstractions
- broad behavioral coverage
- subgoal semantics
- robust temporal decomposition

**There is no massive, diverse “internet of RL experience” comparable to text or image data.**

So classical HRL tried to learn abstractions from data that was often too narrow.

# The Optimization Bias Against Hierarchy

Even if hierarchy is useful, gradient-based learning often prefers shallow solutions.

Typical failure modes:

- options collapse
- one skill dominates
- terminations become too frequent
- manager becomes irrelevant
- low-level policy ignores goals
- high-level actions lose semantics
- flat baseline performs similarly
- training becomes non-stationary

The path of least resistance is often: learn a good flat policy and avoid meaningful hierarchy.

# A Doubt Appears

Why does hierarchy sometimes work so well?

Nachum et al. (2019) [1] asked whether HRL works because of hierarchy itself, or because it improves exploration.

Their conclusion was uncomfortable:

Many empirical benefits could be explained by:

- temporally extended exploration
- better coverage
- smoother action space

Not necessarily by:

- easier policy optimization
- reusable semantic subtasks
- explicit hierarchy as such

[1] “Why Does Hierarchy (Sometimes) Work So Well in Reinforcement Learning?” Nachum et al., 2019.



# Shift #1: From Options to Latent Skills



# From Options to Latent Skills

Classical view:

$$o = (I_o, \pi_o, \beta_o)$$

An option is a temporally extended action with initiation, policy, and termination.

Modern skill-learning view:

$$\pi(a \mid s, z)$$

A skill is often just a latent variable  $z$  that induces a distinguishable behavior.

The abstraction becomes less symbolic and more statistical.

# Skill Discovery as Pretraining

Instead of asking:

How do we decompose **this task**?

ask:

What useful behaviors should exist **before any task**?

Unsupervised skill discovery learns behaviors without external reward.

... Will connect to intrinsic motivation in the next lecture on MBRL

Goal:

- diversity
- coverage
- controllability

Later use:

- exploration
- initialization
- downstream fine-tuning
- skill composition

# DIAYN: Diversity Instead of Reward

DIAYN learns skills that make final states informative about the skill identity.

Skill-conditioned policy:

$$\pi(a \mid s, z)$$

Discriminator:

$$q_{\psi}(z \mid s)$$

Typical objective:

$$\max \mathbb{E} \left[ \log q_{\psi}(z \mid s) - \log p(z) + \alpha \mathcal{H}(\pi(. \mid s, z)) \right]$$

A skill is useful if the state tells us which skill was executed.

# What DIAYN Solves and Does Not Solve

## Solved partially

- discover diverse behaviors
- learn without external reward
- create reusable primitives
- improve exploration

## Still hard

- skills may be diverse but useless
- no task alignment
- no compositional planner
- coverage may not scale

Skill discovery attacks abstraction learning, but does not fully solve subgoal chunking.

# Beyond DIAYN: Geometry of Skills

Later methods tried to make skills less arbitrary.

The key idea:

Latent skill space should reflect controllable change in the world.

Examples:

## DADS:

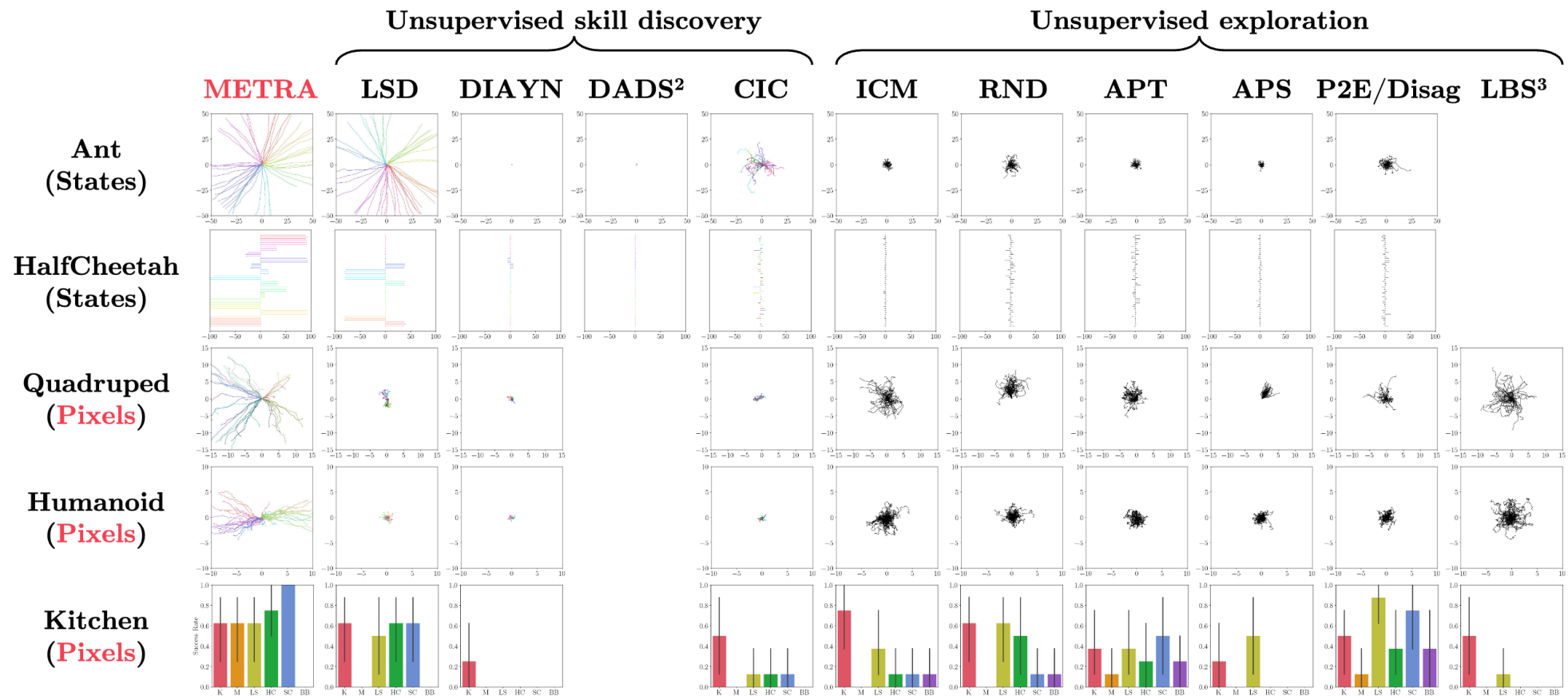
- skills are useful if they induce predictable state transitions
- model  $p(s_{t+1} \mid s_t, z)$

## METRA:

- learn metric-aware abstraction
- cover a compact latent space linked to temporal distance
- $\max < [\varphi(s_{t+1}) - \varphi(s_t)], z >$

DADS: Sharma et al., 2020. METRA: Park, Rybkin, Levine, 2023/2024.

# Learned Skills



## Shift #2: Goal-Conditioned RL as Soft HRL





# GCRL as Soft HRL

Goal-conditioned RL can be read as HRL without explicit options.

HRL interpretation:

- high-level policy chooses goal  $g$
- low-level policy reaches  $g$
- goal persists for several steps

GCRL implementation:

- one policy  $\pi(a \mid s, g)$
- relabeling with HER
- goal sampling curriculum
- learned goal space

The hierarchy is often hidden inside the choice of goals and representations.

# From HIRO to Modern GCRL

HIRO used goals as high-level actions:

$$g_t = s_{t+c} - s_t$$

The worker was trained to reach the goal; the manager was trained to choose useful goals.

Modern GCRL generalizes this such that goals ...

- need not be raw states
- may be latent vectors
- may come from replay
- may come from language
- may be sampled by curriculum
- may be generated by a model

Many “flat” goal-conditioned agents are functionally hierarchical.

# Representation Learning Becomes the Main Workhorse

Earlier HRL already knew that hierarchy depends on abstraction.

The modern shift just moves the burden.

Classical HRL:

- design levels
- define options or goals
- train controllers

Modern view:

- learn representation first
- learn distances / reachability
- plan or condition in latent space

If  $\varphi(s)$  has the right geometry, subgoals become easier to express.

# Abstraction and Chunking

From earlier...

The two hard problems reappear in every modern direction.

Representation problem:

$$s \rightarrow z = \varphi(s)$$

What information should be preserved?

What states are close in the sense of control?

Chunking problem:

$$(s_t, \dots, s_{t+k}) \rightarrow c$$

What sequence segment should be treated as one meaningful behavior?

Different modern paradigms solve different parts of this decomposition.

# World Models: Hierarchy Inside Latent Planning

World models introduce another route.

Instead of learning hierarchy directly in the environment:

$$s_t \rightarrow z_t, \quad z_{t+1} \sim p_{\theta}(\cdot \mid z_t, a_t)$$

learn a latent dynamics model, then plan or control at different scales.

High-level:

- choose latent goal
- plan abstract transition
- maximize task / exploration reward

Low-level:

- realize latent goal
- operate from pixels / observations
- provide grounded control

Examples: Hierarchical World Models, Director, PEG-like hierarchical world-model approaches.

# Director: A Modern HRL-Like World Model

Director learns hierarchical behavior inside a learned world model.

High-level policy:

- selects latent goals
- operates on longer horizon
- plans in model space

Low-level policy:

- achieves latent goals
- acts from observations
- learns grounded behavior

This looks like HRL, but the main engine is model-based representation learning.

Director: “Deep Hierarchical Planning from Pixels”, Hafner et al., 2022.

# Offline RL and Sequence Modeling

When RL is treated as sequence modeling, hierarchy becomes implicit.

Trajectory:

$$\tau = (s_0, a_0, r_0, \dots, s_T, a_T, r_T)$$

Sequence model:

$$p(a_t \mid s_{\leq t}, a_{< t}, r_{\leq t}, \text{condition})$$

Explicit HRL:

- choose subgoal
- execute subpolicy

Sequence view:

- condition on trajectory context
- reuse temporal patterns

Examples: Decision Transformer, Trajectory Transformer etc.

# What Sequence Models Change

We no longer need to label the hierarchy explicitly.

The model can learn patterns such as:

- common action chunks
- repeated subtrajectories
- implicit plans
- task-conditioned behavior
- long-context dependencies
- demonstrations as structure

The hierarchy is a statistical regularity in trajectories.

*And it's much simpler to learn such regularities if you have a direct access to the past through self-attention.*



# The LLM / VLM Era

LLMs changed the role of hierarchy more than its mechanisms.

They often act as the high-level policy:

instruction → plan → subgoals → low-level controller

High-level:

- language reasoning
- task decomposition
- tool selection
- memory use

Low-level:

- RL policy
- motion planner
- controller
- environment-specific executor

# Why LLMs Can Do This

Classical HRL tried to learn decomposition from narrow RL tasks.

LLMs learned decomposition from a different angle.

RL-based HRL:

- interaction data
- sparse rewards
- narrow environments
- embodied trial-and-error

LLM pretraining:

- massive text data
- human reasoning traces
- symbolic problem solving
- descriptions of plans and procedures

LLMs behavior-clone human symbolic reasoning about problems.

... Broadly speaking, that's what we wanted to learn with HRL solutions in the first place

# Imported Hierarchy

Pretrained models do not solve HRL in the classical sense.

They import structure learned elsewhere.

Classical goal:

- discover hierarchy through RL
- learn subgoals from reward
- train end-to-end

Modern practical recipe:

- use pretrained planner
- ground it with perception
- execute with RL / control

The burden shifts from RL training to grounding, verification, and execution.

# Symbolic High-Level, Grounded Low-Level

This revives an old HRL idea in a new form.

Old version:

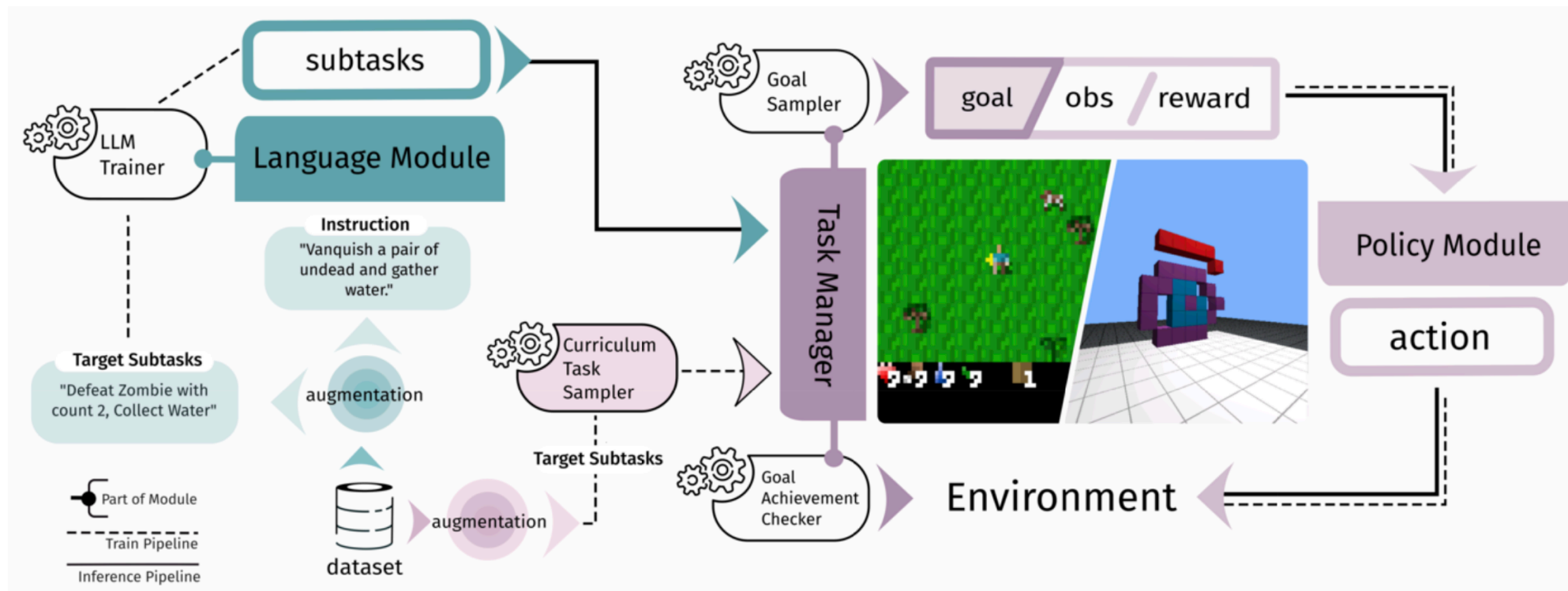
- symbolic planner
- learned controller
- hand-designed interfaces

New version:

- LLM / VLM planner
- learned low-level policy
- language / vision interface

The manager is not only trained by RL, but also pretrained on broad symbolic and multimodal data.

# Naive approach: LLM planner + RL solvers



Paper: Instruction following with goal-conditioned reinforcement learning in virtual environments

# This Does Not Eliminate HRL Problems

The same bottlenecks return in new forms.

Abstraction:

- what should the LLM output?
- language goal?
- object goal?
- latent goal?

Chunking:

- how long should each subtask be?
- when to replan?
- how to recover from failure?

Other problems:

- grounding language in perception and action
- verifying subgoal feasibility
- credit assignment across planner and controller
- distribution shift between pretraining and environment

**TL;DR**



# When Does HRL Actually Help?

Hierarchy is not automatically useful.

It helps when the environment has reusable multi-scale structure.

Good cases:

- sparse rewards
- long horizons
- compositional tasks
- repeated subtasks
- transfer across goals

Weak cases:

- dense reward
- short horizon
- simple locomotion objective
- no reusable subtasks
- flat policy is enough

HRL is an inductive bias. It helps only when the bias matches the task.

..many real world problems do



# A Modern Map of HRL

## Explicit hierarchy:

- options
- feudal policies
- manager / worker

## Skill hierarchy:

- DIAYN
- DADS
- METRA

## Goal hierarchy:

- UVFA
- HER
- HIRO-like agents

## Model / sequence hierarchy:

- world models
- trajectory models
- LLM planners

These are not separate stories. They are different ways of introducing temporal abstraction.

# Three Conceptual Shifts

## 1. Explicit hierarchy → implicit hierarchy

Early HRL had visible levels. Modern methods often hide hierarchy in latent spaces, goals, or sequence context.

## 2. Task decomposition → behavioral primitives

The focus moved from decomposing one task to learning reusable behavior blocks.

## 3. End-to-end HRL → hybrid systems

Modern systems split planning, perception, memory, and control across different pretrained and learned components.

# What Happened to HRL?

HRL identified real structural problems:

- long horizons
- sparse rewards
- exploration
- temporal abstraction
- skill reuse
- compositionality
- planning
- multi-scale behavior

But the solutions did not stay inside one paradigm.

HRL both succeeded and dissolved.

It succeeded because its core ideas became central.

It dissolved because practical solutions now live in representation learning, unsupervised RL, world models, sequence modeling, and LLM-based agents.